

DSA - Avaliação: Recursividade, Merge Sort e Quick Sort

Turma 4 | Valor: 10,0 pontos | Tempo sugerido: 60 minutos | Avaliação individual

Uso adequado de IA

É permitido utilizar ferramentas de IA como apoio durante a avaliação. A ferramenta deve auxiliar a aprendizagem, a execução e a conferência, e não substituir a leitura, a interpretação e o raciocínio. Confira qualquer resposta produzida por IA com os vetores, códigos e estados apresentados no enunciado antes de registrá-la.

Orientações

É permitido consultar o material da disciplina e executar os códigos. Algumas questões dependem de resultados obtidos anteriormente. Não altere os dados da avaliação, exceto quando a própria questão solicitar. Os trechos de código estão em fonte monoespacia e podem ser selecionados para execução e testes.

Resumo de slicing em Python

Expressão	Resultado / significado
<code>vetor[2]</code>	elemento do índice 2
<code>vetor[-1]</code>	último elemento
<code>vetor[2:5]</code>	índices 2, 3 e 4
<code>vetor[:4]</code>	do início até o índice 3
<code>vetor[4:]</code>	do índice 4 até o final
<code>vetor[:]</code>	cópia superficial da lista inteira
<code>vetor[::2]</code>	do início ao fim, passo 2
<code>vetor[::-1]</code>	lista em ordem inversa
<code>vetor[-3:]</code>	os três últimos elementos
<code>vetor[:-3]</code>	todos, exceto os três últimos
<code>vetor[-5:-2]</code>	intervalo usando índices negativos
<code>vetor[6:2:-1]</code>	percurso regressivo do índice 6 até antes do 2

No slicing, a forma geral é **vetor[início:fim:passo]**. O índice de início é incluído e o índice de fim não é incluído. Quando início, fim ou passo são omitidos, o Python assume valores adequados ao sentido do percurso. Por isso, **vetor[:meio]** obtém a parte anterior a *meio*, enquanto **vetor[meio:]** obtém a parte a partir de *meio*. Essa propriedade é usada diretamente no Merge Sort para dividir uma lista em duas partes. O passo também pode ser negativo: **vetor[::-1]**, por exemplo, percorre a lista do fim para o início. Importante: um slicing produz uma nova lista; **vetor[:]** é uma cópia superficial da lista original, não apenas outro nome apontando para o mesmo objeto.

Questão 1 - O caminho até o caso-base (1,0 ponto)

Execute o código abaixo e acompanhe a sequência de chamadas.

```
n = 28

def reduzir(n):
    print(n)
    if n <= 1:
        return
    reduzir(n // 2)

reduzir(n)
```

Q1.A Complete a sequência impressa: 28 . ____ . ____ . ____ . ____

Q1.B Quantas chamadas da função são realizadas, incluindo o caso-base? ____

Considere agora a segunda função:

```
n = 28

def reduzir2(n):
    if n <= 1:
        return
    reduzir2(n // 2)
    reduzir2(n // 2)

reduzir2(n)
```

Q1.C A estrutura das chamadas de **reduzir2** é:

- A) cadeia $n \rightarrow n/2 \rightarrow n/4 \rightarrow \dots$
- B) árvore binária, com duas chamadas menores em cada nível
- C) cadeia $n \rightarrow n-1 \rightarrow n-2 \rightarrow \dots$
- D) sequência crescente $n \rightarrow n+1 \rightarrow n+2 \rightarrow \dots$

Questão 2 - Dividindo o problema (1,0 ponto)

Use o vetor abaixo e considere o trecho de Merge Sort apresentado em seguida.

```
V = [10, 4, 8, 2, 9, 3, 7, 5]

def merge_sort(lista):
    if len(lista) <= 1:
        return lista

    meio = len(lista) // 2
    esquerda = lista[:meio]
    direita = lista[meio:]

    esquerda = merge_sort(esquerda)
    direita = merge_sort(direita)
    return merge(esquerda, direita)

merge_sort(V)
```

Aplique somente a fase de divisão até obter listas de dois elementos.

Q2.A Nível 1 - esquerda: _____

Q2.B Nível 1 - direita: _____

Q2.C Nível 2 - 1ª lista: _____

Q2.D Nível 2 - 2ª lista: _____

Q2.E Nível 2 - 3ª lista: _____

Q2.F Nível 2 - 4ª lista: _____

Q2.G Classifique cada afirmação como V ou F:

- a) Os valores armazenados determinam o ponto em que a lista é dividida. ____
- b) As quatro listas do nível 2 possuem dois elementos. ____
- c) Nenhum elemento precisou ser ordenado para realizar essas divisões. ____
- d) Nesse nível, o caso-base $\text{len}(\text{lista}) \leq 1$ já foi alcançado. ____

Questão 3 - Descida e retorno da recursão (1,0 ponto)

Considere o vetor isolado abaixo. O código executa o Merge Sort sobre apenas essa parte da entrada.

```
V = [10, 4, 8, 2]

def merge(esquerda, direita):
    resultado = []
    i = 0
    j = 0

    while i < len(esquerda) and j < len(direita):
        if esquerda[i] <= direita[j]:
            resultado.append(esquerda[i])
            i += 1
        else:
            resultado.append(direita[j])
            j += 1

    resultado.extend(esquerda[i:])
    resultado.extend(direita[j:])
    return resultado

merge_sort(V)
```

Q3.A merge([10], [4]) = _____

Q3.B merge([8], [2]) = _____

Q3.C merge(resultado de Q3.A, resultado de Q3.B) = _____

Associe: A = divisão; B = caso-base; C = merge.

Q3.D Acontece durante o retorno da recursão: _____

Q3.E É atingido quando a lista possui um único elemento: _____

Q3.F Predomina durante a descida das chamadas: _____

Q3.JUSTIFICATIVA Explique brevemente como a descida e o retorno aparecem nessa execução.

Questão 4 - Acompanhe merge() (1,0 ponto)

Use os dois vetores abaixo. Eles já estão ordenados individualmente.

```
esquerda = [1, 6]
direita = [3, 5]

def merge(esquerda, direita):
    resultado = []
    i = 0
    j = 0

    while i < len(esquerda) and j < len(direita):
        if esquerda[i] <= direita[j]:
            resultado.append(esquerda[i])
            i += 1
        else:
            resultado.append(direita[j])
            j += 1

    resultado.extend(esquerda[i:])
    resultado.extend(direita[j:])
    return resultado

print(merge(esquerda, direita))
```

Q4.A Passo 1: compare ____ × ____; copie ____; i=____; j=____; resultado=_____

Q4.B Passo 2: compare ____ × ____; copie ____; i=____; j=____; resultado=_____

Q4.C Passo 3: compare ____ × ____; copie ____; i=____; j=____; resultado=_____

Q4.D Qual lista terminou primeiro? _____

Q4.E Qual elemento ainda precisa ser acrescentado? _____

Q4.F Resultado final: _____

Questão 5 - Quando uma condição é quebrada (1,5 pontos)

Agora a lista da esquerda não está ordenada. Execute o mesmo merge() da questão anterior.

```
esquerda = [6, 1]
direita = [3, 5]

print(merge(esquerda, direita))
```

Q5.A Primeira comparação: ____ × ____ → copia ____

Q5.B Segunda comparação: ____ × ____ → copia ____

Q5.C Qual lista termina primeiro? _____

Q5.D Elementos restantes: _____

Q5.E Resultado produzido: _____

Q5.F Escolha a conclusão compatível com a execução:

- A) merge() ordena corretamente quaisquer duas listas recebidas.
- B) merge() pressupõe que as duas listas recebidas já estejam ordenadas.
- C) merge() depende da escolha de um pivô.
- D) merge() só funciona quando as duas listas possuem o mesmo tamanho.

Q5.JUSTIFICATIVA Use o resultado produzido pelo código para sustentar a alternativa escolhida.

Questão 6 - Agora mude o mecanismo: Quick Sort (1,0 ponto)

Use o vetor abaixo. O Quick Sort adotará o último elemento da lista atual como pivô.

```
V = [10, 4, 8, 2, 9, 3, 7, 5]

def quick_sort(lista):
    if len(lista) <= 1:
        return lista

    pivo = lista[-1]
    menores = []
    iguais = []
    maiores = []

    for elemento in lista:
        if elemento < pivo:
            menores.append(elemento)
        elif elemento == pivo:
            iguais.append(elemento)
        else:
            maiores.append(elemento)

    resultado_ordenado = quick_sort(menores) + iguais + quick_sort(maiores)
    return resultado_ordenado

quick_sort(V)
```

Q6.A Primeiro pivô: ____

Q6.B menores = _____

Q6.C iguais = _____

Q6.D maiores = _____

Q6.E len(menores) = ____

Q6.F len(maiores) = ____

Q6.G Escolha a afirmação correta:

- A) O Quick Sort divide sempre pela posição central da lista.
- B) Os elementos são distribuídos conforme sua relação com o pivô.
- C) O Quick Sort sempre produz duas partições de mesmo tamanho.
- D) O pivô é removido antes que os elementos sejam comparados.

Questão 7 - Continue a recursão do Quick Sort (1,0 ponto)

A maior partição obtida na questão 6 é armazenada isoladamente em P1. Execute a próxima chamada do Quick Sort sobre ela.

```
P1 = [10, 8, 9, 7]
quick_sort(P1)
```

Antes de considerar o resultado final, acompanhe apenas a primeira partição dessa chamada.

Q7.A P1 = _____

Q7.B Novo pivô = _____

Q7.C menores = _____

Q7.D iguais = _____

Q7.E maiores = _____

Q7.F Complete a trajetória do tamanho do maior ramo: $8 \rightarrow ____ \rightarrow ____$

Q7.G Escolha a afirmação compatível com a execução:

- A) Cada chamada recursiva reduz necessariamente o problema pela metade.
- B) O tamanho dos próximos subproblemas depende da partição produzida pelo pivô.
- C) O Quick Sort não utiliza recursividade.
- D) A distribuição dos valores da lista não interfere nas partições.

Q7.JUSTIFICATIVA Use os tamanhos das listas obtidas em Q7.C e Q7.E para explicar sua escolha. Indique qual delas gera a próxima chamada recursiva de maior tamanho e compare esse tamanho com o da lista P1. A justificativa deve mostrar, a partir desta execução, por que a redução do problema não precisa ocorrer pela metade.

Questão 8 - IA como hipótese, não como resposta (1,5 pontos)

Uma ferramenta de IA sugere: “Experimente usar o valor 8 como primeiro pivô para obter uma divisão mais equilibrada.” Teste a hipótese no vetor V.

```
V = [10, 4, 8, 2, 9, 3, 7, 5]
pivo_teste = 8

def particionar(lista, pivo):
    menores = []
    iguais = []
    maiores = []

    for elemento in lista:
        if elemento < pivo:
            menores.append(elemento)
        elif elemento == pivo:
            iguais.append(elemento)
        else:
            maiores.append(elemento)

    return menores, iguais, maiores

print(particionar(V, pivo_teste))
```

Q8.A menores = _____

Q8.B iguais = _____

Q8.C maiores = _____

Q8.D Diferença absoluta entre $\text{len}(\text{menores})$ e $\text{len}(\text{maiores})$ usando pivô 8: _____

Q8.E Na questão 6, com pivô 5, essa diferença foi: _____

Q8.F Escolha a conclusão sustentada pelos dados:

- A) O pivô 5 produziu a primeira partição mais equilibrada neste vetor.
- B) O pivô 8 produziu a primeira partição mais equilibrada neste vetor.
- C) Os dois pivôs produziram partições igualmente equilibradas.
- D) O uso do pivô 8 garante complexidade $O(n \log n)$ para qualquer entrada.

Q8.JUSTIFICATIVA A sugestão da IA foi confirmada? Use os tamanhos das partições como evidência.

Questão 9 - Teste a conclusão em outra entrada (1,0 ponto)

Considere o novo vetor **W**. Compare duas escolhas de primeiro pivô usando a mesma função `particionar()`.

```
W = [3, 6, 9, 12, 15, 18, 21, 24]

pivo_A = W[-1]
pivo_B = 15

print("Estratégia A:", particionar(W, pivo_A))
print("Estratégia B:", particionar(W, pivo_B))
```

Estratégia A: usar o último elemento como primeiro pivô.

Q9.A Primeiro pivô: ____

Q9.B menores = ____

Q9.C maiores = ____

Q9.D Se essa escolha se repetir em listas já ordenadas, a recursão tende a formar:

A) uma árvore aproximadamente equilibrada.

B) uma cadeia $n \rightarrow n-1 \rightarrow n-2 \rightarrow \dots$

C) uma cadeia $n \rightarrow n/2 \rightarrow n/4 \rightarrow \dots$

D) nenhuma estrutura recursiva.

Estratégia B: usar 15 como primeiro pivô

Q9.E menores = ____

Q9.F iguais = ____

Q9.G maiores = ____

Q9.H Qual estratégia produz a primeira partição mais equilibrada em **W**? ____

Q9.JUSTIFICATIVA Compare os tamanhos das partições das duas estratégias e relacione o resultado ao formato esperado da recursão.

Entrega:

Um arquivo .TXT contendo APENAS AS RESPOSTAS.

Identifique corretamente cada questão.

O nome do arquivo deve estar no padrão <rm_aluno>.txt

Exemplo: 123456.txt